CO-1

ASSESSMENT TOOL 1

APPLICATION-BASED QUESTIONS

NAME:Y.SAI PRASANTH VARMA REGISTRATION NUMBER:192411191

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
<input type="text" name="name">
<input type="email" name="email">
<button>Submit</button>
</form>
```

Questions:

1. Identify appropriate HTML5 semantic elements missing in the structure.

- <header>
- <main>

- <section>
- <footer>
- <label>

2. Modify the structure to make it standards-compliant.

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Online Symposium Registration</title>

</head>

<body>


<h1>College Symposium Registration</h1>


<form action="success.html" method="POST">


    <label>Name:</label><br>

    <input type="text" name="name" required><br><br>


    <label>Email:</label><br>

    <input type="email" name="email" required><br><br>


    <button type="submit">Register</button>


</form>


</body>
```

</html>

3. Interpret the HTTP request generated during form submission.

When the user clicks the Register button, the browser sends an HTTP POST request to the server because the form uses method="POST".

Interpretation:

- **POST** – Sends the form data to the server.
- **/success.html** – The destination specified in the action attribute.
- **HTTP/1.1** – HTTP protocol version.
- **Host: localhost** – The server handling the request.
- **Content-Type** – Indicates that the form data is URL-encoded.
- **Request Body** – Contains the submitted values (name and email).

4. Analyze the HTTP response cycle from server to browser.

The HTTP response cycle describes what happens after the browser sends the HTTP request.

Steps:

1. The user fills in the registration form.
2. The browser sends an HTTP POST request to the server.
3. The server receives and processes the request.
4. The server validates and stores the registration data (if applicable).
5. The server sends an HTTP response back to the browser.
6. The browser receives the response and displays the Registration Successful page.

5. Suggest improvements for usability and accessibility.

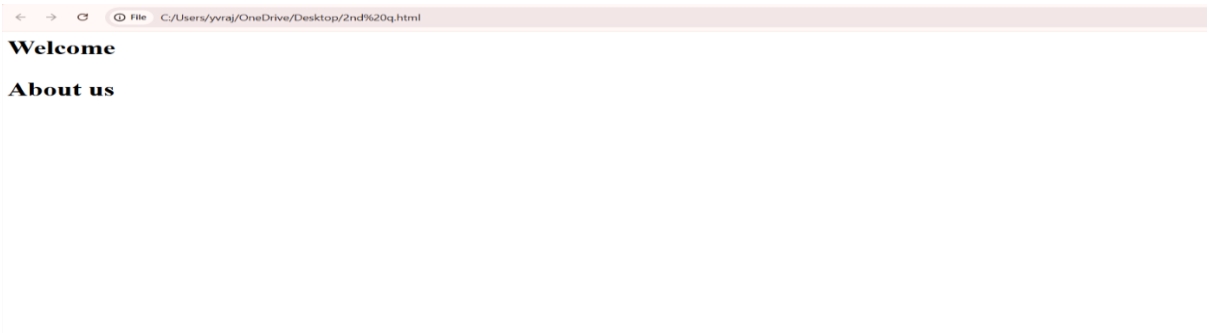
- Use <label> for all input fields.

- Add the required attribute for mandatory fields.
- Use meaningful placeholder text.
- Provide clear error and success messages.
- Ensure keyboard navigation support.
- Use proper color contrast for readability.
- Make the page responsive using the viewport meta tag.
- Use semantic HTML5 elements (<header>, <main>, <section>, <footer>).
- Add ARIA attributes where needed for screen readers.
- Use HTTPS to ensure secure data transmission.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
<head>
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```



Questions:

1. Identify improper nesting and missing closing tags.

Improper Nesting:

- `<p>` is placed inside `<h1>` without closing the `<h1>` tag.

Missing Closing Tags:

- `</h1>`
- `</p>`

2. Analyze how different browsers may interpret this structure.

Different browsers may automatically correct the invalid HTML in different ways. As a result:

- The page layout may appear differently across browsers.
- The `<h1>` and `<p>` elements may be rendered differently.
- Text formatting and spacing may become inconsistent.
- The webpage may not follow the intended structure, causing cross-browser compatibility issues.

3. Identify missing semantic elements (e.g., header, section).

- <header>
- <main>
- <section>
- <footer>

4. Provide a corrected W3C-compliant HTML structure.

```
<!DOCTYPE html>

<html lang="en">

<head>

   <meta charset="UTF-8">

   <meta name="viewport" content="width=device-width, initial-scale=1.0">

   <title>Company</title>

</head>

<body>



<header>

   <h1>Welcome</h1>
```

```
</header>


<main>
   <section>
      <p>About Us</p>
   </section>
</main>


<footer>
   <p>&copy; 2026 Company</p>
</footer>


</body>
</html>
```

5. Suggest techniques to ensure cross-browser compatibility.

- Use valid HTML5 and W3C-compliant code.
- Close all HTML tags properly.
- Use proper HTML5 semantic elements.
- Test the webpage in multiple browsers (Chrome, Firefox, Edge, Safari).
- Use external CSS and avoid browser-specific features unless necessary.
- Validate the HTML and CSS before deployment.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

<form action="/submit" method="GET">

<input type="text" name="username">

<input type="password" name="password">

<button type="button">Login</button>

</form>

Questions:

1. Identify issues in the form configuration.

- method="GET" should be changed to POST for login forms.
- <button type="button"> should be <button type="submit"> to submit the form.
- <label> elements are missing.
- required attributes are missing for input fields.

2. Analyze why the HTTP request is not triggered.

- The HTTP request is not triggered because the button type is button, not submit.
- A button with type="button" does not submit the form.
- Changing it to type="submit" sends the HTTP request to the server when clicked.

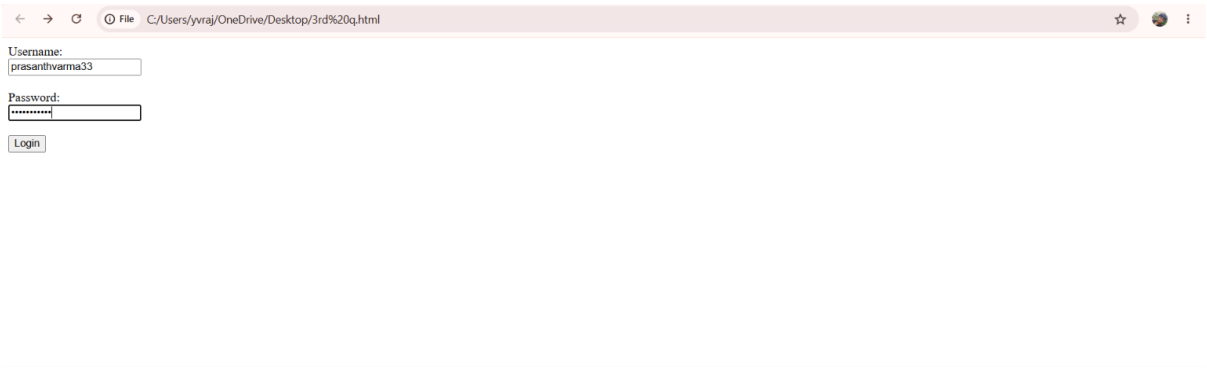
3. Interpret the role of GET method in this scenario.

Interpretation:

- The GET method sends form data as URL parameters.
- The submitted data is visible in the browser's address bar.
- It is suitable for retrieving data, not for login forms because sensitive information (such as passwords) can be exposed.
- POST is the preferred method for authentication and sensitive data.

4. Propose a corrected implementation.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login Form</title>
</head>
<body>

<form action="/submit" method="POST">

    <label for="username">Username:</label><br>
    <input type="text" id="username" name="username" required>
    <br><br>

    <label for="password">Password:</label><br>
    <input type="password" id="password" name="password" required>
    <br><br>

    <button type="submit">Login</button>

</form>

</body>
</html>
```



5. Suggest improvements for secure data transmission.

Improvements:

- Use HTTPS instead of HTTP.
- Use the POST method instead of GET.
- Mark all required fields with the `required` attribute.
- Validate user input on both the client and server.
- Encrypt sensitive data during transmission using SSL/TLS.